Variables – types, const and readonly

variables

- Class variables – These are static variables. They are self initialized

```
public class A{
     static int a;  //automatically initialized to 0.
     public void func(){
          Console.WriteLine(A.a);
     }
}
```

- Member variables – variables that are not static and not a method variable. They are self initialized to default values. Int is 0 and bool is false.

```
public class A{
     int a;  //automatically initialized to 0.
     public void func(){
          A obj = new A();  //this instantiation initializes all the non static members.
          Console.WriteLine(A.a);
     }
}
```

variables

- Local variables – these are non static member function variables.

```
public class A{
     public void func(){
          int a = 0;  //must be explicitly initialized
          Console.WriteLine(a);
     }
}
```

variables

```
public class Program {
        private static int x = y + 10;
        private static int y = x + 5;
        public static void Main(string[] args) {
                Console.WriteLine(Program.x);
                Console.WriteLine(Program.y);
                Console.ReadKey();
        }
}// 10, 15
```

C# always initializes static variables to their initial value after creating them. Variables x and y are therefore given a default of zero value. C# now realizes that these variables declared need to be assigned some values. C# does not read all the lines at once but only one at a time. It will now read the first line and as the variable y has a value of 0, so x will get a value of 10. Then at the next line, y is the value of x + 5. The variable x has a value of 10 and so y now becomes 15. As C# does not see both lines at the same time, it does not notice the circularity of the above definition.

variables

```
public class Program {
        private int x = y + 10;
        private int y = z + 5;
        private int z = 0;
        public static void Main(string[] args) {
                Program obj = new Program();
                Console.WriteLine(obj.x);
                Console.WriteLine(obj.y);
                Console.ReadKey();
        }
}// Error
```

It does not work for instance variables as the rules of an instance variable are quite different than that of static variables. The initializer of an instance variable has to be determined at the time of creation of the instance. The variable y does not have a value at this point in time. It can't refer to variables of the same object at the time of creation. So we can refer to no instance members to initialize an instance member.

constant

We need to initialize the const variable at the time we create it. We are not allowed to initialize it later in our code or program.

```
public class Program {
        private const int x = y + 100;
        private const int y = z - 10;
        private const int z = 300;
        public static void Main(string[] args) {
                        System.Console.WriteLine("{0} {1} {2}",x,y,z);
                Console.ReadKey();
        }
}//390 290 300 . Shocked?
```

C# is very smart to realize the dependency among the constants here.

constant

const variables cannot be circular, i.e., they cannot depend on each other in circle.

```
public class Program {
        private const int x = y + 100;
        private const int y = z - 10;
        private const int z = 300;
        public static void Main(string[] args) {
                System.Console.WriteLine("{0} {1} {2}",x,y,z);
                Console.ReadKey();
        }
}//Error:  The evaluation of the constant value for 'AccessModifiers.Program.x' involves a circular definition
```

constant

- A constant by default is static and hence we cannot use a reference variable to initialize it. Also since it is already static, we cannot make a constant static.

```
public class ClassA {
        public const int aaa = 10;
}
public class Program {
        public static void Main(string[] args) {
                ClassA classA=new ClassA();
                Console.WriteLine(classA.aaa);
                Console.ReadKey();
        }
}//Error:cannot access const field using reference
```

```
public class ClassA {
        public static const int aaa = 10;
}
public class Program {
        public static void Main(string[] args) {
                ClassA classA=new ClassA();
                Console.WriteLine(classA.aaa);
                Console.ReadKey();
        }
}//Error a const cannot be static
```

constant

A const field of a reference type other than string can only be initialized with null.

```
public class ClassA { }
public class Program {
        public const ClassA classA=new ClassA();
        public static void Main(string[] args) {
        }
}//Error
```

```
public class ClassA { }
public class Program {
        public const ClassA classA=null;
        public static void Main(string[] args) {
        }
}//works
```

A const is a variable whose value once assigned cannot be modified, but its value is determined at compile time only. The error disappears as we now initialize classA to an object which has a value that can be determined at compile time i.e., null. We can never change the value of classA, so it will always be null. Normally, we do not have consts as classA reference type as they have value only at runtime. new() actually gets executed at runtime and therefore does not get value at compile time. So this results in an error.

readonly

Readonly is similar to const
(value cannot be changed once set)
but with subtle changes.

Const is compile time constant.
Readonly is runtime constant.
Const is implicitly static but readonly
is not implicitly static.

For a readonly field, assignment
to the field can only occur as
part of the declaration or in a
constructor in the same class.

```
public class Program {
    public readonly int a = 10 ;
    public readonly int b;
    public readonly int c;
    public Program(){
        b = 10;
        c = 10;
    }
    public Program(int x){
        b = x;
        c = x;
    }
    public static void Main(string[] args) {
    Program obj = new Program();
    Console.WriteLine(obj.a); //10
    Console.WriteLine(obj.b); //10
    Console.WriteLine(obj.c); //10

    Program obj1 = new Program(20);
    Console.WriteLine(obj1.a);//10
    Console.WriteLine(obj1.b);//20
    Console.WriteLine(obj1.c);//20
     }
}
```

readonly

- Readonly struct – the readonly keyword on a struct declares that the struct is immutable. Every instance field of the struct must be marked readonly.

```
public readonly struct Point
{
    public double X { get; }
    public double Y { get; }

    public Point(double x, double y) => (X, Y) = (x, y);

    public override string ToString() => $"({X}, {Y})";
}
```

or

```
public readonly struct Point
{
    public readonly double X;
    public readonly double Y;

    public Point(double x, double y) => (X, Y) = (x, y);

    public override string ToString() => $"({X}, {Y})";
}
```

Use of readonly

- A potential use of readonly fields can be to use the config settings, that will be fetched during runtime and not be changed throughout the program life cycle.

App.config | Program.cs*

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <appSettings>
    <add key="PI" value="3.14"/>
  </appSettings>
</configuration>
```

```
class Program
{
    public static const int cmToMeters = 100; // Compile time constant
    public static readonly double PI; // Runtime constant
    public static Program()
    {
        PI = Convert.ToDouble(ConfigurationSettings.AppSettings[0]);
    }
    static void Main(string[] args)
    {

    }
}
```

Note: Both readyonly and const variables cannot be passed by references (i.e. by ref or out). This is because a ref or out argument must be an assignable variable.

var

Local variables can be declared without giving an explicit type. The `var` keyword instructs the compiler to infer the type of the variable from the expression on the right side of the initialization statement. The inferred type may be a built-in type, an anonymous type, a user-defined type, or a type defined in the .NET Framework class library.

```
var i = 5;
var a = new[] { 0, 1, 2 };
var anon = new { Name = "Terry", Age = 34 };
```

- var can only be used when variables are declared and initialized in the same statement. They cannot be initialized to null.
- var can only be used on local variables (variables declared at method levels)
- Multiple implicitly-typed variables cannot be initialized in the same statement.